python workshop plan test

Gavin Klorfine

Installation (might be headache to get everyone installed properly)

(Three?) options:

1. VS Code + Conda
    - I think this makes the most sense. Seems to be the standard.
    - Probably do not need to spend much time with Conda; can set them up using the Navigator or just provide a "cheat sheet" of terminal commands
2. PyCharm + Conda
    - Another option instead of VS Code.
3. PsychoPy standalone
    - Makes installation super easy, but the standalone platform leaves much to be desired.
    - Likely good enough for a workshop; however, anyone serious about Python will just end up having to learn the above on their own :/

Very Basics

Variable = space in memory

Basics of variable types

1. `int` (integer; "whole number")
    - less memory
    - 1 is an `int`
2. `float` (real; "decimal number")
    - 1.2 is a `float`

- 1.0 is a `float`

3. `str` (string; characters/words with no numerical meaning)
    - `"Hello world!"` is a string
    - `"416-676-6767"` is also a string (does it make sense to add two phone numbers together?)
4. `bool` (boolean; can be `True` or `False`; more on this later)
5. `None`
    - A special type of variable …

```
x = 1       # Assign value to a variable

type(x)     # Check variable type

str(x)      # Convert x to string

print(x)    # Print x to console
```

```
1
```

The above examples `type()`, `str()`, and `print()` are all examples of functions …

```
x = 1
x = 3   # Variables can be updated. Now 3 is stored in a space labelled x

# What will the below display?
print(x)
type(x)
```

```
3
```

```
int
```

More advanced variable types

Multiple values can be stored in one variable. A list is one way to do this:

```
x = [0, 1, 2, 3, 4]     # Define a list; square brackets
print(type(x))

print(x[0])                     # Index first element of a list
print(x[-1])                    # Index last element
print(x[0:2])                   # first, second (but not third)
print(x[0:1])                   # another (silly) way to index first element

print(len(x))                   # length of list
print(x[len(x) - 1])            # Another way to find the last element (not so silly!)
```

```
<class 'list'>
0
4
[0, 1]
[0]
5
4
```

Explain indexing (0-based; [inclusive, exclusive)) ...

List methods (not sure how many to cover)

```
x = [0,1,2,3,4]

x.append(5)     # Append (add) entry to list
print(x)

x.extend([6,7,8,9])   # Extend a list by "glueing" it to another list
```

```
[0, 1, 2, 3, 4, 5]
```

Difference b/w append and extend illustrated

```
x = [0,1,2,3,4]
y = [0,1,2,3,4]


x.append([0,1,2,3,4])
```

```
y.extend([0,1,2,3,4])

print(x)
print(len(x))

print(y)
print(len(y))
```

```
[0, 1, 2, 3, 4, [0, 1, 2, 3, 4]]
6
[0, 1, 2, 3, 4, 0, 1, 2, 3, 4]
10
```

Tuples

Lists are mutable, can be changed, whereas tuples are immutable (elaborate)

```
x = (3, 4, 5)   # Define a tuple; round brackets
print(type(x))

x.append(2) # Error
```

```
<class 'tuple'>

AttributeError: 'tuple' object has no attribute 'append'
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[50], line 4
      1 x = (3, 4, 5)   # Define a tuple; round brackets
      2 print(type(x))
----> 4 x.append(2) # Error
AttributeError: 'tuple' object has no attribute 'append'
```

Some useful properties...

```
# Unpacking a tuple
x, y = (50, 70)

print(x)
print(y)
```

```
50
70
```

Strings

Some more details on strings...

```
cringe = "Hello world!"

cringe[0:5]
```

```
'Hello'
```

They are kind of like lists in that they are *iterable* (elaborate)

They are kind of like tuples in that they are *immutable*

```
cringe = "Hello world!"
cringe[0:6] = "Goodbye" # Error
```

```
TypeError: 'str' object does not support item assignment
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[53], line 2
      1 cringe = "Hello world!"
----> 2 cringe[0:6] = "Goodbye" # Error
TypeError: 'str' object does not support item assignment
```

Therefore, there are methods to "modify" them if needed (they just create a new string "behind the scenes")

```
cringe = "Hello world!"

print(cringe.replace("Hello", "Goodbye"))   # Replace first entry with the second

meme_list = "baby gronk, 67, anime betrayal, skibidi toilet, the rizzler"

print(meme_list.split(","))     # Split on ","; returns a list

print(meme_list.split(",")[0])  # Combine previous knowledge to get baby gronk ö

# How would you index "the rizzler"? Can you think of more than one way?
```

```
Goodbye world!
['baby gronk', ' 67', ' anime betrayal', ' skibidi toilet', ' the rizzler']
baby gronk
```

To include variables in text:

```
x = 3.14159
print(f'Approximate value of pi is {x}') # note the f and curly braces

print(f'To three decimal places is {x:.3f}') # .3f for 3 decimal place float
```

```
Approximate value of pi is 3.14159
To three decimal places is 3.142
```

Back to boolean

Basics of boolean logic:

A boolean variable (type `bool`) can be either `True` (`1`) or `False` (`0`).

Comparison operators

```
x = True    # Define a boolean variable

print(x == False)  # Check (==) to see if x is False.
print(x != False)  # is x NOT False (i.e., is x True)?

y = 40

print(y < 50)   # True
print (y > 50)  # False

print (y < 40)  # False (they are equal)
print (y <= 40) # True
```

```
False
True
True
False
False
True
```

Mathematically, the == operator is more similar to a traditional equals sign than = is in Python (elaborate or exclude).

Logical operators

AND:

- True AND True == True
- True AND False == False
- False AND True == False
- False AND False == False

OR:

...

```
# Can show above in python maybe
```

Conditional statements

While loops

For loops

Importing packages

```
# Some packages included by default
import math

print(f'the value of pi is {math.pi:.3f}')

# Some (usually) require some work with conda
import numpy as np      # Common convention

x = np.random.normal(loc = 0, scale = 1, size = 1000)   # sample of 1000 random values from s
```

```
the value of pi is 3.142
```

Defining functions

```
def addition(a, b):
    return a + b

print(addition(a = 1, b = 5))

def subtraction(a, b = 1):  # Give b default value of 1
    return a - b

print(subtraction(a = 5))
```

```
6
4
```

Modules / .pkl files?

Numpy

Pandas

Matplotlib

Basic stats (t-tests, correlation, ...)